

Alexander Ludwig

MW 12-2

[Home](#) [Homework](#)

Unit 1 Test Review Questions

Due: 3/13/26

Description

This HW will serve as a review of the material thus far and have questions to prepare you for the upcoming exam. "Unit 1: Computers and Binary" was centered on introducing the language of the computer and the layers of translation that occur within it. These topics will be the primary focus along with the material online it is associated with:

- Computers Generally
 - [Introduction](#)
- The Abstraction Hierarchy
 - [Introduction](#)
 - [C Compilation Lab](#)
- The C Language
 - [C Review](#)
 - [C Compilation Lab](#)
- Binary and Hexadecimal Numbers
 - [Binary and Hex](#)
- Logical and Bitwise Operations

The test will have True/False, Multiple Choice, Short Answer, Fill-in-the-Blank, and Number Conversion Type Questions. They will all be based on these short answer questions. Answer each to the best of your ability and use the material and in-class notes to fill in any gaps you find you have. This is meant to be a graded review, simply looking up answers without understanding will not prepare you for the exam, try your best and ask questions during class time.

Section 1: Computers Generally - /30 pts

1. (4 pts) | ~4-5 sentences What are the two main classifications of computers historically? What differentiates them? Which are used today?

The most common classifications of computers are analog and digital. A computer generally is known as "a thing that computes". Analog are built with a specific purpose, meaning they only have one thing they can compute. They also do not store any memory. Whereas modern digital computers are reprogrammable and therefore have multiple purposes. They also have storage to have persistent data. Digital computers are much more common nowadays, including anything from a calculator to servers.

2. (4 pts) | ~3-4 sentences What is a Turing Machine generally? Was one made? Why or why not?

A Turing machine is a theoretical computer that is capable of computing anything that is computable by existing instructions from memory. It consists of an infinite tape of memory, a read/write head and a state machine that follows instructions to manipulate the data in the tape. One has never been created due to the physical limitations of its existence, mainly the concept of an infinite tape of memory.

3. (4 pts) | ~3-4 sentences With respect to memory in C, can you see how Turing Machines conceptually influenced the way we interact with memory? How do they relate?

A Turing machine uses an infinite tape for memory reading step-by-step left to right. This influences how we think of memory in C because when assigning addresses to variables and pointers in C, the addresses can be visually seen as linear, designating one portion of memory after the next. Also, when parsing an array, it can be compared to moving through the tape in a Turing machine.

4. (4 pts) | ~3-4 sentences Explain the Abstraction Hierarchy inside of a computer. Where do consumers mostly work? Where do we work?

The abstraction hierarchy demonstrates the layers at which computation takes place. Broken down into ten layers, the majority of consumers only interact with the top of the hierarchy, the application layer. Then developers interact with the following layers developing algorithms in programming languages which is broken down into assembly and machine code during compilation, each of which has its own layer.

5. (6 pts - 1.5 pts per step) In the C Compilation Lab, we saw how C is compiled from source code to an executable. Map each step of the compilation process to a layer in the abstraction hierarchy.

Step	Description	Abstraction Layer
Preprocessing	C preprocessor formats source code using macros like #define or #include	Programming Lang.
Compilation	Translates C source code into assembly code	Assembly Lang.
Assembly	Labeling assembly into binary machine code	Instruction Set Arc.
Linking	orders object files and combines them into a single .exe file	Instruction Set Arc.

6. (4 pts) | ~3-4 sentences Transistors are the core of modern digital computers. What are their states? How many are there? How does that inform our number system on the computer?

Transistors have two states, on and off. This maps directly to the binary values of 0 and 1. This allows computers, which only use binary, to use Boolean algebra and algorithms gates within circuits to actually compute.

7. (2 pts) | ~2-3 sentences In the CPU of the Von Neumann architecture, what are the two main components? What is the relationship between them?

There are the arithmetic logic unit and the control unit within the CPU. The CU decodes the instructions and manages data flow while the ALU performs the actual arithmetic and logical operations.

8. (2 pts) | ~2-3 sentences When running code on a computer, is it run as assembly code?

No, during compilations assembly code is converted into machine code (binary). This allows the computer to actually execute functions since it only understands 0s and 1s.

Section 2: Binary and Hexadecimal - /40 pts

1. (4 pts - 1 pt each) How big is a `char`, `int`, `float`, and `double` in memory in C?

Type	Size (bytes)	Size (bits)
<code>char</code>	1	8
<code>int</code>	4	32
<code>float</code>	4	32
<code>double</code>	8	64

2. (3 pts) | ~2-3 sentences What are number systems? Which do we use normally? Which do computers use?

A number system is a way of representing numbers via digits or symbols. We most often use decimal or base-10, which consists of digits 0-9 for each number place. Whereas computers read and write binary which only consists of 0 and 1 for each number place.

3. (3 pts) | ~2-3 sentences Binary and Hexadecimal are names for number systems, what are their bases? What do we use hex for mostly?

Binary has base-2 hence the prefix bin- and hexadecimal, prefix hex- represents a base-16 system. Hex is most commonly used for a more readable version of binary, RGB values, and MAC addresses.

4. (3 pts) | ~1-2 What is a bit, nibble, and byte? How big is each? What are their maximum unsigned values?

Unit	Size	Max Unsigned Value
Bit	1 bit	0b1
Nibble	4 bits	0b1111
Byte	8 bits	0b11111111

5. (3 pts) | ~3-4 sentences What are MSB and LSB? How did we use it in HW1 to manipulate the bit-plane of an image?

MSB is used to represent the most significant bit. Also known as the greatest value represented in a binary number (furthest left). The LSB is just the opposite. The far right bit only holding a max decimal value of 1. In hw1 we used bit masking to change grayscale and rgb values at the bit level in images to reveal watermarks, and extract individual colors and strengths.

6. (5 pts - 1 pt each) Convert the following decimal numbers into binary:

Decimal	Binary
23	10111
100	1100100
3	11
255	11111111
127	1111111

7. (5 pts - 1 pt each) Convert the following binary numbers to decimal:

Binary	Decimal
111	7
11111111	255
00000001	1
10101	21
10110111	183

8. (6 pts - 1 pt each) Convert the following binary numbers into hex digits:

Binary	Dec	Hex
1100	12	C
1111	15	F
11000001		C1
01100111		67
11011011		DB
1011111011101111 B F		BEEF

Section 3: Logical and Bitwise Operations - /30 pts

1. (2 pts) What are the 4 main logical operators?

AND, OR, XOR, NOT

2. (2 pts) | ~1-2 sentences What is a truth table?

A truth table is a visual representation of all possible outputs and inputs for a given function.

3. (8 pts - 2 pts each) Create 4 truth tables showing the 4 main logical operations.

AND

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

OR

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

NOT

A	NOT A
0	1
1	0

XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

4. (14 pts - 2 pts each) Find the resulting number from these operations. First convert the number from decimal to binary, then apply the operation bitwise. Show your work.

1. $3 \& 7 =$ $\begin{array}{ccc} 011 \\ 111 \end{array} \rightarrow 011$
2. $2 | 7 =$ $\begin{array}{ccc} 010 \\ 111 \end{array} \rightarrow 111$
3. $\sim(15) =$ $\begin{array}{cccc} 01111 \\ 10000 \end{array}$
4. $21 \wedge 10 =$ $\begin{array}{ccccc} 10101 \\ 01010 \end{array} \rightarrow 11111$
5. $2 \ll 1 =$ $10 \rightarrow 100$
6. $64 \gg 2 =$ $100000 \rightarrow 001000$
7. $(3 \& 7) | (2 \ll 1) =$ $\begin{array}{ccc} 011 \\ 100 \end{array} \rightarrow 111$

5. (2 pts) | ~1-2 sentences + expression Design a bitwise check to see if a number is even or odd.

A bitwise check for an even or odd binary number can be reduced down to $(bin \gg 1) \& 1$ yields true $\rightarrow$ bin is odd

6. (2 pts) | ~1-2 sentences + expression If a graphics program organizes its colors into RGB pixels represented by single numbers in hex, like `0xRRGGBB`. Write a bitwise equation to carry over the red and blue channels and max the green channel out.

For each pixel, $newColor = oldColor | 0x00FF00$. This preserves R and B values while making G always maxed.

Point Summary

Section	Points
Section 1: Computers Generally	/30
Section 2: Binary and Hexadecimal	/40
Section 3: Logical and Bitwise Operations	/30
Total	/100